AWS CLOUD INTERNSHIP PROJECT REPORT

Serverless Image Processing Service Using AWS Lambda and Amazon S3

1. Introduction

Cloud computing provides scalable and flexible resources for developing and deploying applications without the need to maintain physical servers. Serverless computing is one of the important cloud computing approaches in which developers can execute application logic without managing the underlying infrastructure.

As part of the AWS Cloud internship project, a serverless image processing service was developed using Amazon S3 and AWS Lambda. The system automatically processes images whenever a new image is uploaded to an Amazon S3 bucket.

The implemented system performs image processing operations such as resizing, compression, and watermarking. The processed image is then stored in a separate S3 bucket. AWS Lambda performs the processing only when an image-upload event occurs, eliminating the need for continuously running servers.

The project demonstrates the practical use of AWS serverless services, object storage, event-driven architecture, IAM permissions, and Python-based image processing.

2. Problem Statement

Traditional image processing applications may require servers or virtual machines to continuously run application code. Managing such infrastructure increases operational complexity and can result in unnecessary resource consumption when image processing is not continuously required.

Therefore, there is a need for an automated and scalable image processing system that can process images whenever they are uploaded, without requiring an always-running server.

The proposed system addresses this problem by using Amazon S3 as the image storage service and AWS Lambda as the serverless image processing service.

3. Objectives

To develop a serverless image processing system using AWS services.

To store uploaded images using Amazon S3.

To automatically trigger an AWS Lambda function whenever an image is uploaded.

To resize images automatically using Python and Pillow.

To compress images to reduce their file size.

To add a configurable text watermark to processed images.

To store processed images in a separate Amazon S3 bucket.

To use IAM for controlling access to AWS resources.

To implement the complete system without requiring an always-running EC2 server.

To demonstrate an event-driven and pay-per-use cloud architecture.

4. Proposed System

The proposed system consists of two Amazon S3 buckets and an AWS Lambda function.

The first S3 bucket is used for uploading original images. When an image is uploaded, an S3 Object Create event automatically triggers the Lambda function.

The Lambda function receives information about the uploaded image from the S3 event. It downloads the image into memory and processes it using the Pillow library.

The processing includes resizing the image when its width exceeds the specified maximum width, adding a text watermark, converting the processed image to JPEG, compressing the image using a specified JPEG quality, and uploading the result to a separate S3 bucket.

The processed file is stored with the prefix **processed-**.

5. System Architecture

User → Upload Image → Amazon S3 Upload Bucket → S3 Object Creation Event → AWS Lambda Function → Pillow Image Processing (Resize, Watermark, Compress) → Amazon S3 Processed Bucket → Processed Image

6. Technologies Used

Technology / Service	Purpose
AWS Lambda	Serverless image processing
Amazon S3	Image storage and event triggering
AWS IAM	Access control and permissions
Python 3.12	Lambda programming runtime
Pillow	Image processing
boto3	Communication with AWS services
AWS CloudWatch	Lambda execution logging

7. Software and Hardware Requirements

Software Requirements

- AWS account
- AWS Management Console
- Python 3.12 runtime
- AWS Lambda
- Amazon S3
- AWS IAM
- Pillow
- boto3

Hardware Requirements

The project does not require a dedicated server or high-end computing system because the image processing is performed using AWS Lambda. A computer with an internet connection and a web browser is sufficient for

configuring and testing the application through the AWS Management Console.

8. Implementation

8.1 Creating S3 Buckets

Two S3 buckets are used: an upload bucket for original images and a processed bucket for images after processing. Separating the input and output buckets helps organize the original and processed files.

8.2 Creating the IAM Role

An IAM execution role is created for AWS Lambda. The role provides the Lambda function with permissions required to access objects in Amazon S3, read uploaded images, write processed images, and generate execution logs. The project uses the AmazonS3FullAccess policy and AWSLambdaBasicExecutionRole according to the implemented configuration.

8.3 Creating the Lambda Function

A Lambda function is created using Python 3.12. The configured memory is 256 MB and the timeout is 30 seconds. The Lambda function contains the image-processing logic. The environment variable DESTINATION_BUCKET specifies the bucket where processed images are stored.

8.4 Adding Pillow to Lambda

The Pillow library is required for image manipulation. Since Pillow is not directly included in the basic Lambda runtime, it is added using a Lambda layer. A Klayers Lambda layer is used to provide the Pillow dependency.

8.5 Configuring the S3 Trigger

An S3 trigger is added to the upload bucket and configured for All object create events. Whenever an image is uploaded to the input bucket, Amazon S3 automatically invokes the Lambda function.

9. Image Processing Workflow

Step 1: Image Upload

The user uploads an image to the designated S3 upload bucket.

Step 2: Event Generation

Amazon S3 detects the object creation and generates an event.

Step 3: Lambda Invocation

The S3 event automatically invokes the AWS Lambda function.

Step 4: Reading the Event

The Lambda function obtains the uploaded bucket name and object key from the S3 event.

Step 5: Downloading the Image

The uploaded image is downloaded into memory using boto3.

Step 6: Resizing

If the image is wider than 800 pixels, it is proportionally resized.

Step 7: Watermarking

A configurable text watermark is added to the bottom-right area of the image.

Step 8: Compression

The processed image is converted to JPEG and compressed using JPEG quality 70.

Step 9: Uploading

The processed image is uploaded to the destination S3 bucket using the processed- prefix.

10. Lambda Function Logic

- Receives the S3 event.
- Extracts the bucket name and object key.
- Downloads the uploaded image from S3.
- Loads the image using Pillow.
- Checks whether resizing is required.
- Resizes the image proportionally when necessary.
- Adds the configured watermark.
- Converts the image to JPEG.
- Compresses the image using JPEG quality 70.
- Uploads the processed image to the destination S3 bucket.

The implementation uses boto3, the AWS SDK for Python, to communicate with Amazon S3.

11. Key Configuration Parameters

Parameter	Value / Purpose
Lambda Runtime	Python 3.12
Lambda Memory	256 MB
Lambda Timeout	30 seconds
Maximum Image Width	800 pixels
JPEG Quality	70
Input Storage	S3 Upload Bucket
Output Storage	S3 Processed Bucket
Output Prefix	processed-
Processing Library	Pillow
AWS SDK	boto3

12. Testing

The system was tested by uploading an image to the configured S3 upload bucket.

The expected sequence during testing was: **Image Upload** → **S3 Event** → **Lambda Invocation** → **Image Processing** → **Processed Image Upload**.

After successful processing, the processed image appears in the destination S3 bucket. The output image is resized when necessary, contains the configured watermark, and is compressed as a JPEG image.

13. Results

- Images can be uploaded to an S3 bucket.
- Upload events automatically trigger the Lambda function.
- Images are processed without requiring an EC2 instance.
- Images wider than 800 pixels are resized proportionally.
- A text watermark is added to the image.
- Images are converted and compressed as JPEG files.
- Processed images are stored in a separate S3 bucket.
- The complete workflow operates using an event-driven serverless architecture.

14. Advantages

14.1 Serverless Architecture

There is no requirement to maintain a continuously running server.

14.2 Automatic Processing

Image processing begins automatically when an image is uploaded.

14.3 Scalability

AWS Lambda can execute the processing function when required without requiring manual server management.

14.4 Reduced Infrastructure Management

The project does not require EC2 instances or manually maintained servers.

14.5 Cost Efficiency

The project uses serverless, pay-per-use AWS services and is designed to operate within the AWS Always Free tier under the stated usage limits.

14.6 Automated Workflow

The complete process from image upload to processed-image storage is automated.

15. Cost Analysis

The project is designed using AWS services that have Always Free tier allowances. According to the project implementation:

- AWS Lambda: 1 million free requests per month under the applicable Always Free allowance.
- Amazon S3: Free-tier storage and limited free GET/PUT requests under the applicable allowance.
- EC2: Not required.
- Load Balancer: Not required.

Since the application does not use continuously running EC2 instances or a load balancer, unnecessary infrastructure costs are avoided. Actual charges can depend on AWS account eligibility, region, usage, and current AWS pricing.

16. Security and Access Control

AWS IAM is used to control access to AWS resources. The Lambda function operates using an IAM execution role. This role provides the permissions needed for S3 operations and CloudWatch logging. Using IAM roles avoids embedding AWS credentials directly into the Lambda function.

17. Challenges

- Configuring two S3 buckets for input and output.
- Connecting the S3 upload event to Lambda.
- Providing the correct IAM permissions.
- Adding the Pillow dependency through a Lambda layer.
- Configuring Lambda memory and timeout.
- Managing the destination S3 bucket through an environment variable.
- Ensuring the processed image is correctly uploaded to the destination bucket.

These configurations are important because the application depends on the correct interaction between multiple AWS services.

18. Future Scope

- Supporting multiple image-processing operations.
- Allowing users to configure the desired image dimensions.
- Supporting additional output formats.
- Adding different watermark styles.
- Creating an API-based interface for image uploads.
- Adding authentication for controlled access.
- Adding monitoring and notification features.
- Extending the system to process larger numbers of images automatically.
- Adding metadata extraction and storage.
- Integrating the processed images with a web or mobile application.

19. Conclusion

The Serverless Image Processing Service Using AWS Lambda and Amazon S3 successfully demonstrates how cloud-based serverless technologies can be used to automate image processing.

The system uses Amazon S3 to store uploaded images and generate object-creation events. These events automatically invoke an AWS Lambda function, which processes the image using Python and Pillow. The image can be resized to a maximum width of 800 pixels, watermarked, converted to JPEG, and compressed using a quality value of 70. The processed image is then stored in a separate S3 bucket.

The project demonstrates important AWS concepts including serverless computing, object storage, event-driven architecture, IAM permissions, Lambda layers, and AWS SDK integration.

Overall, the project provides a practical example of building an automated cloud application without requiring continuously running servers and highlights the benefits of AWS Lambda and Amazon S3 for lightweight, event-driven processing tasks.

20. References

1. AWS Lambda documentation.
2. Amazon S3 documentation.
3. AWS IAM documentation.
4. AWS boto3 documentation.
5. Pillow Python Imaging Library documentation.
6. AWS CloudWatch documentation.
7. AWS Lambda Layers documentation.